

A Solution for the Naming Problem for Name-Centric Services

Torsten Teubler, Mohamed A. Hail, and Horst Hellbrück

Lübeck University of Applied Sciences, Electrical Engineering and Computer Science,
Mönkhofer Weg 239, 23562 Lübeck, Germany
{teubler,hail,hellbrueck}@fh-luebeck.de
<http://www.cosa.fh-luebeck.de>

Abstract. In recent past name-centric or content-centric networking (CCN) has gained substantial attention in the networking community. In a further development step name-centric service architecture enables the flexible placement and distribution of services in the network especially in a heterogeneous environment of wired and wireless (sensor) networks. However, the problem of structuring and creating hierarchies for names in name-centric networks is not solved yet. E.g. there is no configuration of service names in name-centric service WSN, no concept of unsolicited names or link-local names in CCN. In IP networks, DHCP or IPv6 auto-configuration is available, but no equivalent technique exists for CCN. We analyze the naming problem in the software development life cycle for name-centric services in WSN and propose a structure, hierarchy, and configuration mechanism for names. The paper introduces the overall concept and preliminary steps of implementation.

Keywords: Content-Centric Networking, Wired/Wireless Environments, Service Development, Service Management

1 Introduction

The Internet has evolved in the last decades from a network of devices with a handful of standardized protocols and applications to a service platform for numerous applications. In the recent past the focus of the Internet has evolved from devices to services and content delivery. Today's Internet business is concentrated on content and applications running on wireless devices.

The challenge for the future development of the Internet is a seamless integration of sensor networks that provide content and services where the devices delivering these services are not of importance at all. Therefore, new paradigms like content-centric networking (CCN) and name-centric networking (NCN) are proposed for an alternative architecture of the future Internet [8]. CCNx is a reference implementation for content-centric networking from Palo Alto Research Center that received attention in the CCN community [1]. CCNx provides a powerful API and a well-designed core engine called daemon. In a recent paper

we have presented a way to develop name-centric services with CCNx and a lightweight CCN implementation for wireless sensor networks (WSN) [6]. However, one of the most basic problems for applications and services in CCN is not solved. CCNx neither provides a name configuration mechanism nor suggests a consistent naming scheme.

In this paper, we address the naming problem in a structured way from the design and development of services, via the deployment of these services on the platform until the startup of the services during runtime. Different requirements and naming parts need to be assigned during this software development life cycle.

In summary, the contributions of the paper are as follows:

- We analyze the naming problem for name-centric services in the software development life cycle for name-centric services in mixed wired wireless topologies.
- We propose a structure and hierarchy for names and introduce unsolicited names or link-local names in a naming concept.
- We present the usage of alias names and mapping at gateways at the boundaries between wired and wireless topology.
- We describe the overall concept and implementation details.

The rest of the paper is organized as follows: Section 2 presents related work for name-centric approaches and auto-configuration mechanisms. Section 3 presents background information and motivates the naming problem more specific. In Section 4 we perform a structured analysis of the naming problem and develop the solution. Section ?? presents some implementation details with focus on the gateway between wired and wireless topologies. The gateway provides a name configuration and a name alias service. The paper concludes with a summary and a discussion of future work.

2 Related Work

This section first discusses related work on name or content-centric approaches. In the second part, we introduce automatic configuration approaches in Id-centric networks as they are related to our configuration approach for name-centric services.

A well-known approach of content-centric networking for WSN is Directed Diffusion [7]. Directed Diffusion uses an addressing scheme tightly coupled to the application. Therefore, a standardized addressing convention is hard to implement with Directed Diffusion. The same holds for SPIN [9], another content-centric networking protocol for WSN. It uses also an addressing scheme tightly coupled to the application.

The authors of “Service-Centric Networking” [3] propose a consistent naming convention for services and content. They give examples of content and service names but do not classify the parts of the name. Configuration mechanisms are not part of this work.

In the recent past, CCNx [8, 1] gained attention. With CCN-WSN [11] we developed a CCNx derivate for WSN and wireless networks. In CCNx content is addressed by name not by its location or host, where it is stored. CCNx allows a maximum flexibility for names without limitations or semantics of names. However, a semantic meaning of names for name-centric services is desirable.

In Id-centric networking, configuration of addresses and networking parameters is ubiquitous. DHCP [5] is a prominent example for configuration of nodes in a network. The fact that automatic configuration is important in large networks is seen in the further development of the IP protocol. In Version 6 (IPv6) auto-configuration protocol standard [10] nodes integrate automatically in a local network.

In summary, recent approaches in name-centric networking do not cover naming conventions. In our opinion, naming convention is the key for automatic configuration. Our approach targets name-centric services in a mixed wired and wireless topology. To the best of our knowledge, we are the first to present a naming concept and a configuration mechanism for service names in name-centric networks.

3 Name-Centric Services

In this section we introduce CCN and how name-centric services can be implemented with CCN. This introduction is needed to prepare for the discussion of the naming problem in the last subsection.

3.1 CCN

In content-centric networking (CCN) content is addressed by names. We will introduce only the most basic concept of CCN which are needed to understand the naming problem and our solution. More detailed information is available on CCNx technical documentation web page [4] and [11].

CCN follows a hierarchical naming scheme following the URI syntax [2]. It is recommended that the first component is a registered unique DNS name to avoid name clashes among different service providing authorities. Message types in CCN are *interests* and *content objects*. Interests request content by name. Content objects carry service responses to an interest back to the requester.

A CCN node comprises the daemon and its three storages: *Content Store* (CS), *Pending Interest Table* (PIT), and *Forward Information Base* (FIB). The daemon is the protocol processing unit. Applications using CCN communicate via the daemon using *faces*. Faces in CCNx are a generalization of an interface comparable to TCP/IP sockets. When an interest arrives, the daemon looks up the content store if it contains a content object matching the interest name. An interest name matches a content name, if the interest name equals the prefix of the content name and the prefix is the longest prefix among all compared content names (longest prefix match). If no content object matches in the content store, the pending interest table is searched. If an interest in the pending interest table

matches, the incoming face of the recent interest is associated with this entry. If no entry matches, the interest is forwarded to a suitable face and an entry is created in the PIT. Content objects are sent only as a response to an interest. They are stored in a content store and arrive through faces generated by services.

3.2 Services with CCN

CCN is a powerful general purpose protocol that can be the basis for name-centric services. In [6] we introduced a name-centric service architecture. Interests can contain service requests by using so called *command marker* in CCN. Content objects contain service responses from the service provider. The advantage is that with the name-centric service oriented architecture (SOA), transparent access to services is achieved independent of their location (wireless networks or wired network in the Internet/Cloud). Our future vision is an easy-to-use SOA programming paradigm based on a name-centric service engine. For this goal, a standardization of service names is inevitable. Consequently, we propose a solution for this problem in this work.

3.3 The Naming Problem

Names in CCN can be chosen arbitrarily, following institutional or global conventions. The only restriction is that names describing the content must be unique. A typical solution to this restriction is a naming scheme according to the directory structure of a file system. The first name component is a global unique domain name. Further name components are the directory names of a file system. This name hierarchy or folder structure has contextual semantic meaning to users.

Assignment of names for services is different. Different parts of service names must be configured at different phases of the software life cycle which is not yet considered in CCN. During service development names are assigned to service methods by the developer of the service and should reflect the purpose of the method. During the service deployment in the next phase, contextual information needs to be added to the service name describing the context of the service. It is important to stress that the structure of the contextual information in a service name has not been considered in the past.

Thus, CCN does not specify any semantics on names there are no predefined valid names for limited access comparable to *link-local*, *unsolicited*, and *multi- or unicast* addresses. Such predefined addresses are necessary for automatic configuration and are also missing in name-centric networks.

A second severe problem with CCN naming is that names are usually verbose. Verbose names are useful for humans but not for resource constrained wireless devices. Verbose names waste memory on constrained devices and messages sent over wireless links become considerably large wasting bandwidth and energy. Name components can be mapped to shorter name components for example at the transition from the wired to the wireless network. For mapped name components, it is necessary that they preserve the shortlex order of names as

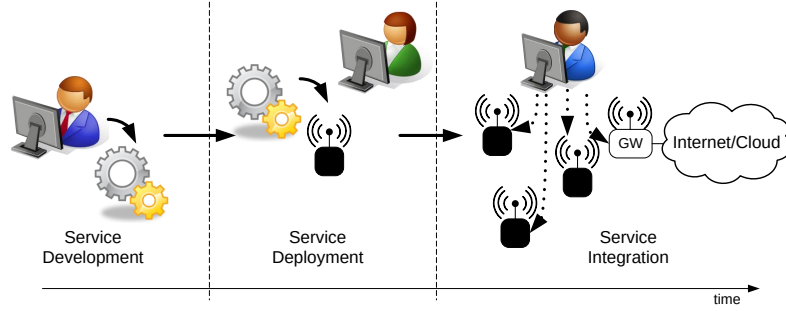


Fig. 1. Life Cycle of Wireless Devices

explained in the CCNx technical documentation [4]. The shortlex order of names is necessary because in CCN a content object with the longest prefix match on the name is returned in response to a request. Well-known compression techniques for strings will not work because they do not preserve shortlex ordering. An important additional requirement is that this mapping must be generated at runtime without reconfiguring whole service names and content object names in the wireless network.

4 Naming Approach for Name-Centric Services

In order to solve the naming problem introduced in Section 3.3 systematically we introduce and analyze the software development life cycle for name-centric services with the focus on service names. Then, we present our naming concept for name-centric services. We conclude this section with a discussion of the necessary architectural changes of the basic architecture of wireless networks integrated in the wired Internet.

4.1 Life Cycle for Name-Centric Services

One can compare the software development life cycle of name-centric services to the software development life cycle of distributed applications running on an office PC of a company. In this life cycle of an application different institutions are involved. First a software manufacturer develops the application software. This software is then deployed by an IT-support company according to the wishes of the end-user. Finally, the end-user integrates the PC with the software in his company network. In summary, the life cycle can be roughly divided into three phases. The same life cycle phases exist for name-centric services in a wireless network, depicted in Figure 1.

During the first phase named *service development* a manufacturer-service developing institution—assigns names for commands or methods of services. The second phase is the *service deployment*. Here, services are deployed to the

wireless devices for example by another institution. Like programs on a PC (e.g. text-processing and spreadsheet are office programs) services on a wireless device can be grouped into categories during this service deployment phase. The third phase is named *service integration*. During this phase, the “end-user” deploys the wireless devices and configures the gateway and thus, integrates the services in the company’s wireless network. Deployment and configuration is depicted as dotted arrows in Figure 1. Each of the phases introduced above will assign parts of a name for our service. In the next section we will introduce a concept that will reflect this assignment of the names in the different phases.

4.2 Naming Concept for Service

In this section we develop the naming concept for services by a dedicated example. The example is based on the following name: `/fh-luebeck.de/building-18/floor-2/room-14` This name addresses services on a sensor node placed on university campus of Lübeck University of Applied Sciences, building 18, second floor, room 14. The first component is a global unique domain name of the institution. Further components address the *site*. The addressing scheme for sites follows an individual institutional naming scheme. In this example the site describes a physical location on a campus but a site can be any logical addressing as well.

As a suffix of the site addressing components we propose one or more components which semantically group the category of services. This can be for example “climate” describing a category of climate related services like temperature and humidity. The last component of the service name is a command beginning with a command marker. For details of command marker we refer to the CCNx technical documentation [4] due to space restrictions here. A command represents a service method (see [6]).

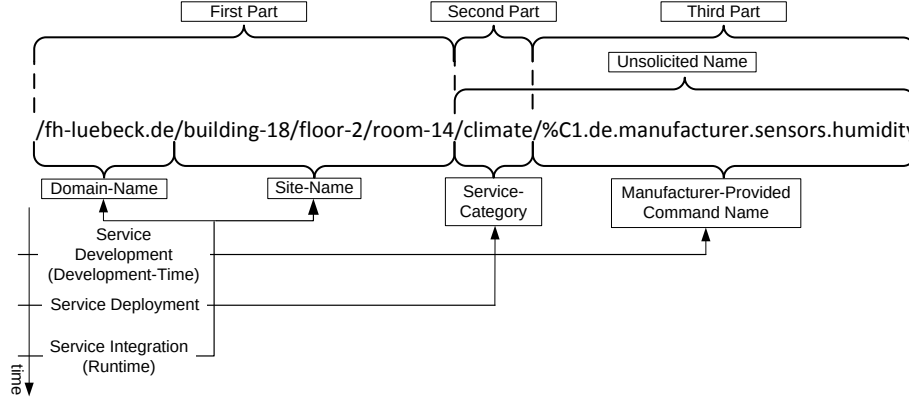
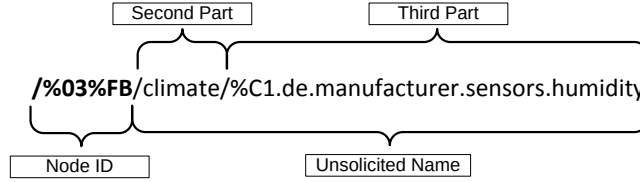
Domain name, site, category of service, and command together form a *fully-qualified service name*. An example of a fully-qualified service name is `/fh-luebeck.de/building-18/floor-2/room-14/climate/%C1.de.manufacturer.sensors.humidity`.

Each life cycle phase (see Figure 1) maps to a part of a service name. Figure 2 shows three parts of the service name and the phase in the life cycle where they are assigned.

During service development the so called *manufacturer-provided service name* is assigned. A manufacturer-service developing institution—assigns names for commands or methods. Manufacturer-provided command names are the *third part* of the service name.

During service deployment, services are deployed on the node and grouped together in categories. These names of the categories are the *second part* of the service name.

With the second and the third part of the name set, services are accessible but only in a local network, not in the name-centric Internet. We term service names with second and third part set *unsolicited service names*. The parts forming the unsolicited service name are shown also in Figure 2. There are maybe many

**Fig. 2.** Example Service Name**Fig. 3.** Example Link-Local Service Name

nodes with the same unsolicited service name in a local network. It is not clear which node answers a request with an unsolicited service name.

To send a request to a node with a certain node-Id in a local network, a so called *link-local name* is necessary. A link local name is the unsolicited service name prefixed by a name component representing the node-Id. An example link-local service name is shown in Figure 3. The node-Id in the first component is highlighted. The so called *link-local name service* on a node answers requests addressed to link-local names. At first glance, link-local names contradict to the name-centric paradigm but nodes are entities like services and therefore, they should be addressable by CCN. Furthermore, link-local names are necessary for configuration and maintenance.

The *first part* of a service name consists of *domain* and *site* name. It is assigned by the “end-user” institution who runs a wireless network. This institution integrates ready-to-use nodes in the company’s wireless network and configures the first part of the service name. A name with first part (domain and site name), second part, and third part set is a fully-qualified service name.

The first component of a link-local name can be seen as an alternative first part of a service name. Note that a link-local name is not a fully-qualified service name. Only services with a fully-qualified service name are accessible from the name-centric Internet.

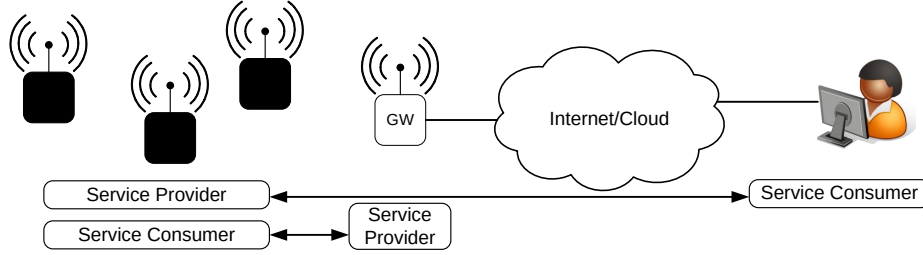


Fig. 4. Overview of the Architecture

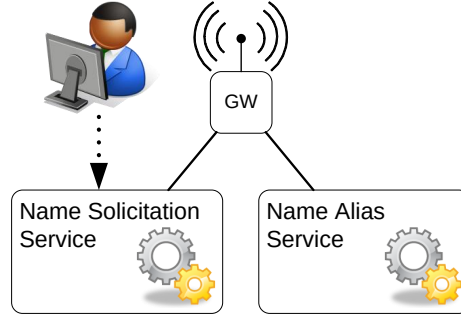


Fig. 5. Gateway Architecture

4.3 Architecture

In the last sections we introduced name-centric services as well as the naming concept for these services with focus on wired and wireless networking topologies. In this part of the work we describe the system architecture for our suggested solution. Figure 4 depicts the wired and wireless devices as well as the name-centric services running on these devices. Wireless devices, for example sensor nodes, provide services to give consumers actual context information about real-world phenomenon via the Internet/Cloud. For the naming-concept suggested in this work the architecture needs to be extended by additional services for configuration and the support for link-local names. We will explain the services for the most important device in the architecture namely the gateway that connects the wired and wireless topology first.

Gateway The gateway (“gw”) provides transition between wired/wireless networks and it provides services to the wireless network (see Figure 4). For this purpose the gateway performs a protocol conversion from the wired to the wireless protocol and vice versa for each service request and service response.

The gateway additionally provides the important configuration services to the name-centric wireless network. The *name solicitation service* provides name configuration for the third phase of our life cycle during the runtime of the name-

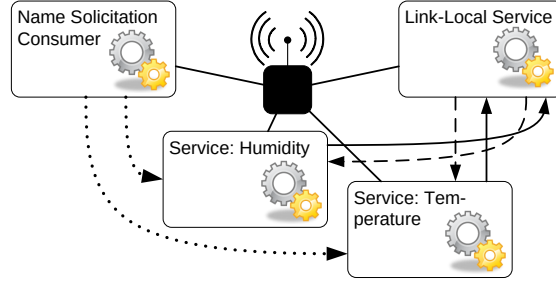


Fig. 6. Sensor Node Architecture

centric service. In Figure 5 the gateway runs such a name solicitation service. Additionally, a *name alias service* for mapping verbose names from the Internet to alias names used in the wireless network topology runs on this gateway. We will explain details of these services in the next section.

Wireless Devices Figure 6 shows the extended architecture of a wireless device. In the middle of Figure 6 the wireless device and around software components like services or service consumers are depicted. This software is executed on the wireless device (indicated by solid lines in Figure 6).

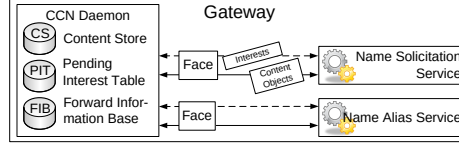
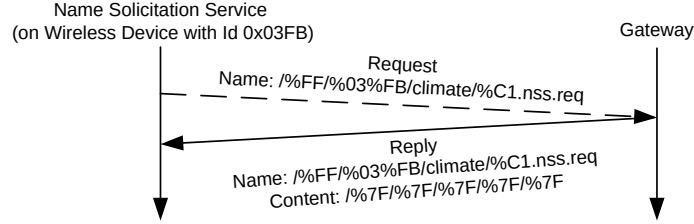
Counter-clockwise the service solicitation consumer, services for sensor data (humidity and temperature), and link-local service are depicted.

The name solicitation service configures the first part of the service names during service integration. Before the wireless devices can be integrated the name solicitation service has to be configured by the institution who operates the wireless network (dotted arrow in Figure 6). Details of the name solicitation service will be explained in Section 5.2.

The link-local service receives service link-local named requests over the network. If the first component matches the node-Id from the wireless device, the link-local service sends a request with the unsolicited service name to the local services (dashed arrows in Figure 6). One of the services on the node responds and sends a response back to the link-local service (solid arrows in Figure 6). The link-local service then adds the node-Id to the response and forwards it on the wireless network.

5 Implementation

In this section we describe some implementation aspects of the naming concept introduced in the previous sections. We start with the gateway implementation. Then, we describe the implementation of the *name solicitation service* and *name alias service* in more detail.

**Fig. 7.** Schematic Overview of the Gateway**Fig. 8.** Message Exchange during Name Solicitation

5.1 Gateway

In contrast to many monolithic approaches our gateway is implemented as modular collection of individual services which provides a high degree of flexibility. All services are implemented as *CCN applications*. CCN applications communicate with each other via the CCN daemon exchanging requests and replies as shown in Figure 7. On the left side the CCN daemon is depicted. On the right side, name solicitation and name alias service are shown as an example.

Our gateway is based on the Java CCNx implementation [1]. Consequently gateway services are implemented in Java where each service is a collection of Java classes. To enable the gateway the CCNx daemon has to be started first and then the services.

The gateway hardware comprises a low power industrial PC with Intel Atom 1.6 GHz CPU, 1 GB DDR2 RAM, and a 30 GB solid state hard disk running Windows operating system. For communication with the wireless sensor nodes the gateway contains an IEEE 802.15.4 transceiver attached to a USB port.

5.2 Name Solicitation Service

The name solicitation service listens for requests sent from the name solicitation consumers in the wireless network. On the gateway the name solicitation service responds with the first part of the service name. Message exchange between a wireless device and the gateway is shown in a path-time diagram in Figure 8.

A name solicitation request from a name solicitation consumer from a wireless device with Id 0x03FB has the name `/ %FF/ %03%FB/ climate/ %C1.nss.req`. Requests from name solicitation consumers are broadcasted through the network. The first component `%FF` is the service name for the name solicitation

service. Then, a component containing the node-Id and a component containing the service category (second part of the service name) follows. The last component is the service method for name requests (**req**) of the name solicitation service (**nss**).

To assign the first part of service name to a service, the name solicitation service needs to know the first part of the service name. The first part is configured in an XML file which is stored in the configuration section of the gateway in a file which is read by the name solicitation service during startup. Listing 1 shows an example XML configuration for two wireless devices. Node 0x03FB runs climate monitoring services and node 0x03FC runs services for light and window control. The grouping of tags—from general-to-specific—follows the form general-to-specific semantics of the service name.

Listing 1. XML Name Solicitation Configuration

```
<Configuration>
  <Domain name="fh-luebeck.de">
    <Site name="building-18">
      <Site name="floor-02">
        <Site name="room-14">
          <LinkLocal name="%03%FB">
            <Category name="climate"/>
          </LinkLocal>
          <LinkLocal name="%03%FC">
            <Category name="light"/>
            <Category name="window"/>
          </LinkLocal>
        </Site>
      </Site>
    </Site>
  </Domain>
</Configuration>
```

With the information the name solicitation service gets from the name solicitation consumer the necessary information can be retrieved from the configuration file. The first part of the name for services under the category “climate” of node 0x03FB is /fh-luebeck.de/building-18/floor-2/room-14.

5.3 Alias Service

Verbose service names are a problem in wireless networks as discussed in Section 3.3. Therefore, a mapping is implemented between the verbose name of the wired topology and an alias name used in the wireless topology. These alias names are only used in the wireless network exclusively. At the gateway the alias service maps verbose names to alias names and vice versa.

A mapping of a verbose name to an alias is stored in a dictionary or hash map. To ensure that the aliases follow the order of the names a simple scheme consisting of numbers can be used as aliases and assigned to the verbose names accordingly. For example, the name “my-verbose-name” maps to alias “1” and name “my-other-verbose-name” maps to alias “2”. In this simple example the aliases follow the shortlex ordering of CCN. This is because in shortlex order “my-verbose-name” comes before “my-other-verbose-name” and “1” comes before “2”.

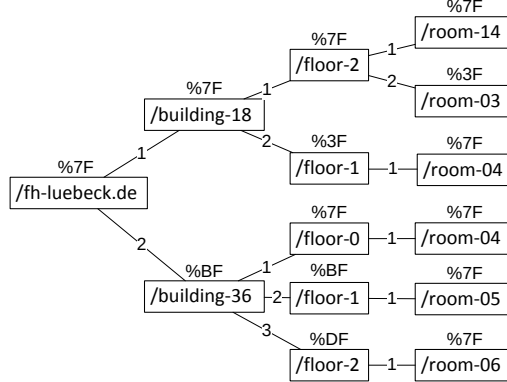
Mapping aliases manually is a cumbersome task. Furthermore, the above presented mapping is not extendable if the aliases should follow name shortlex ordering. For example if one want to add the name “my-verbose-name-two” to the dictionary containing “my-verbose-name” and “my-other-verbose-name”. This name is placed according to the canonical ordering between “my-verbose-name” and “my-other-verbose-name”. The mapping of “my-other-verbose-name” has to be changed to alias “3” because “my-verbose-name-two” now maps to alias “2”.

Therefore, we implemented a more sophisticated algorithm of the alias service, that maps the names to alias numbers independently. It allows adding new aliases while the aliases follow the name shortlex ordering.

This basic algorithm is shown in pseudo code style in Listing 2. To insert a new name in the dictionary `addAlias(name, MAX_NUMBER/2, 0, MAX_NUMBER)` is called, where `MAX_NUMBER` is the largest number assigned for aliases. The dictionary is a collection of tuples (`alias`, `name`).

Listing 2. Algorithm for adding Alias Numbers

```
addAlias(name, piv, lo, up) {
    tempName = dictionary.alias(piv)
    if (tempName == "") {
        dictionary.add(piv, name)
        SUCCEEDED
    }
    if (lo >= up || lo + 1 == up) {
        FAILED: "Number-Range exceeded!"
    }
    if (tempName > name) {
        addAlias(name, (lo + piv)/2, lo, piv)
    } else if (tempName < name) {
        addAlias(name, (up + piv)/2, piv, up)
    } else {
        FAILED: "Name already inserted!"
    }
}
```

**Fig. 9.** Service Name Tree

The algorithm fails, if the range of numbers exceeds. The assignment of the alias numbers is also not very efficient and depends on the sequence the names are inserted into the system.

It is possible to add aliases to each component of a service name independently under consideration of the position in the service name. This works because components of service names follow a tree hierarchy. An example tree hierarchy is shown in Figure 9. An alias—shown as hexadecimal number above each node—is mapped for each component in that tree. Numbers on edges in the tree graph Figure 9 indicate the time the components were configured. For example, service name `/fh-luebeck.de/building-18/floor-2/room-14` maps to `/%7F/%7F/%7F/%7F`.

The name solicitation service requests the aliased service name during name solicitation and assigns the aliased name to the service. If a request from the Internet is received at the gateway, the alias service maps the service name to the shorter alias service name and forwards it to the wireless network.

Advantages of the alias algorithm are that new components can be added without changing the aliases of the other components. This is beneficial if names are configured ad-hoc and the sequence of the names is not known in advance. Furthermore, alias names do not need explicitly configured, and alias names follow the canonical order of the aliased names. One drawback is that only a limited number of names can be aliased.

6 Conclusion and Further Work

In this work, we motivated the need and the advantages of name-centric services based on concepts of content-centric networking (CCN). However, the problem of naming is neither solved for CCN nor for name-centric services. Therefore, we developed and introduced a solution to the naming problem based on an analysis of the software development life cycle of name-centric services for wired

and wireless topologies. The naming concept divides a service name into several name parts which are assigned at different phases of the service development and deployment. The implementation of a name solicitation service running on the gateway between the wired and the wireless topologies shows the configuration mechanism of service names during runtime. Additionally, an alias service maps verbose names to short-length alias names in wireless networks and thereby demonstrates the flexibility and benefits of this naming approach. The name solicitation service is about to be finished on the gateway. In the future we will implement an exemplary application together with a name-centric service engine on our sensor network testbed.

Acknowledgments This work was funded by the Federal Ministry of Education & Research of the Federal Republic of Germany (17PNT017, DataCast). The authors alone are responsible for the content of this work.

References

1. Project CCNx. [Online] <http://www.ccnx.org/>
2. Berners-Lee, T., Fielding, R., Masinter, L.: Uniform resource identifiers (uri): Generic syntax (1998)
3. Braun, T., Hilt, V., Hofmann, M., Rimac, I., Steiner, M., Varvello, M.: Service-centric networking. In: Communications Workshops (ICC), 2011 IEEE International Conference on. pp. 1–6 (2011)
4. CCNx Project: CCNx Technical Documentation (May 2013), <http://www.ccnx.org/releases/latest/doc/technical/>
5. Droms, R.: Dynamic Host Configuration Protocol. RFC 2131 (Draft Standard) (Mar 1997), <http://www.ietf.org/rfc/rfc2131.txt>, updated by RFCs 3396, 4361, 5494, 6842
6. Hellbrück, H., Teubler, T., Fischer, S.: Name-Centric Service Architecture for Cyber-Physical Systems (Short Paper). In: Service-Oriented Computing and Applications (SOCA), 2013 6th IEEE International Conference on. p. tbd (2013)
7. Intanagonwiwat, C., Govindan, R., Estrin, D.: Directed diffusion: a scalable and robust communication paradigm for sensor networks. In: Proceedings of the 6th annual international conference on Mobile computing and networking. pp. 56–67. MobiCom '00 (2000)
8. Jacobson, V., Smetters, D.K., Thornton, J.D., Plass, M.F., Briggs, N.H., Braynard, R.L.: Networking Named Content. In: Proceedings of the 5th international conference on Emerging networking experiments and technologies. pp. 1–12. CoNEXT '09, ACM, New York, NY, USA (2009)
9. Kulik, J., Heinzelman, W., Balakrishnan, H.: Negotiation-based protocols for disseminating information in wireless sensor networks. *Wirel. Netw.* 8(2/3), 169–185 (Mar 2002)
10. Narten, T.: Neighbor discovery and stateless autoconfiguration in IPv6. *Internet Computing, IEEE* 3(4), 54–62 (1999), <http://dx.doi.org/10.1109/4236.780961>
11. Zhong, R., Hail, M.A., Hellbrück, H.: CCN-WSN - a lightweight, flexible Content-Centric Networking Protocol for Wireless Sensor Networks. In: 2013 IEEE Eighth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (IEEE ISSNIP 2013). Melbourne, Australia (Apr 2013)